

CSCI 112

Programming with C

LECTURE: 18

J. L. Pach

OUTLINE:

- Linked Lists:
  - Introduction
  - Creating and Linking Nodes
  - Traversing the List
  - What Happens in the Loop

LINKED

Lists

INTRODUCTION TO LINKED LISTS 1/4

Let's talk about **arrays**. Imagine we have an array designed to store a list of products that a customer wants to buy in an online store. By default, the array has a size of 20, because people usually buy only a few products.

Of course, an edge case must eventually happen — a customer tries to add the **21st product**. What should the program do if the array has only 20 elements?
It cannot simply add the 21st element after the allocated memory.

Why not? Because, as you know, **static memory (on the stack)** is reserved one variable after another — like:

```c
int x, y;
```



INTRODUCTION TO LINKED LISTS 2/4

These memory slots are fixed and continuous. Besides, even if we decided to use **dynamic memory allocation**, we still face a problem: to insert the 21st element, we would need to :

- Create a new array with size larger than the previous one (for example, +1),
- Copy all elements from the old array,
- Add the new element, and
- Free the old memory (if it was dynamically allocated).

> Now, what if our customer wants to add yet another product? As you can see, this approach is **extremely inefficient**.

INTRODUCTION TO LINKED LISTS 3/4

Arrays have a **fixed size**, so one common workaround is to increase the array not by 1, but by a fraction of its currentPtr size — for example, by half. However, the same problem still occurs: the larger the array becomes, the more expensive it is to:

- declare a new array,
- allocate new memory,
- and copy all existing elements.

> So we can ask: *Is there another way to store data whose size changes during program execution, without copying everything from scratch each time?* Yes, there is.

INTRODUCTION TO LINKED LISTS 4/4

The answer actually appeared on this slide already — but as a word in everyday language, not yet as a data structure: we are talking about a list.

Defining a Node: Let's imagine a data structure that has **two fields**:

This structure can store:

- an integer value (`value`),
- a pointer to another structure of the same type (`nextPtr`).

```c
typedef struct Node
{
    int value;
    struct Node *nextPtr;
} Node;
```



Creating and Linking Nodes

Now, let's create an independent pointer to the first element:

I prefer the naming convention `headPtr` rather than `head`, because it clearly emphasizes that `headPtr` does not allocate memory or create a node. It only points to an existing one — just like this analogy:

```c
typedef struct
{
   int value;
   struct Node *nextPtr;
} Node;
```



```c
Node *headPtr;
```

```c
int x = 5;
int *xPtr = &x;   // xPtr does not create a new variable; it only points to x
```



Creating and Linking Nodes

```c
typedef struct{
    int value;
    struct Node *nextPtr;
} Node;
```



```c
int x = 5;
int *xPtr = &x;
```



Through `xPtr`, we can modify `x` itself, for example: `*xPtr = 2;` The same idea applies to `headPtr`. Let's create our first node:

```c
Node node_1;
node_1.value = 5;         // or shorter: Node node_1 = {5, NULL};
node_1.nextPtr = NULL;    // currentPtrly, our node does not point to anything
```



Now let's create a **second** node:

```c
Node node_2 = {7, NULL};
```



We can now **connect** our elements into a list:

```c
headPtr = &node_1;        // same idea
node_1.nextPtr = &node_2;  // as with &x
```



TRAVERSING THE LIST

- Now we can move through the list using the pointer `headPtr` and read the elements just like we did with an array. For an array, we might do:

```c
for (int i = 0; i < n; i++)
{
  printf("%d\n", arr[i]);
}
```

- For our list, we can do:

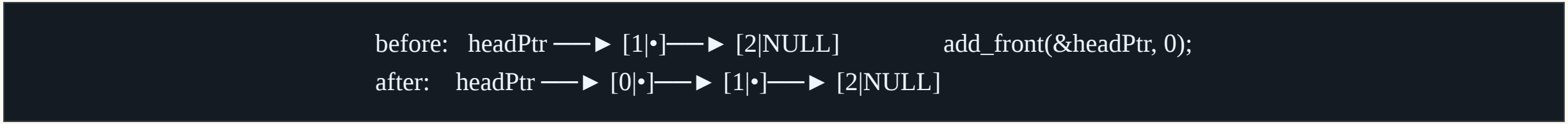
```c
for (Node *currentPtr = headPtr; currentPtr != NULL; currentPtr = currentPtr->nextPtr)
{
  printf("%d\n", currentPtr->value);
}  //Node *currentPtr = headPtr;  a copy of the pointer to the first element
```

Adding a Node at the Beginning 1/2

- The new node must live **after** the function returns, so we allocate it on the **heap** (`malloc`).
- The function has to change the caller's `headPtr`, so it receives the **address** of that pointer (`Node **`) – the same idea as `safe_free(void **ptr)` .

```c
bool add_front(Node **headPtrPtr, int value)
{
    Node *newNodePtr = malloc(sizeof *newNodePtr);
    if (newNodePtr == NULL)
    {
        return false;                   // allocation failed, list unchanged
    }
    newNodePtr->value = value;
    newNodePtr->nextPtr = *headPtrPtr;      // 1. the new node points to the old first node
    *headPtrPtr = newNodePtr;               // 2. head now points to the new node
    return true;
}
```

Adding a Node at the Beginning 2/2



- **Order matters:** if you did step 2 first, you would lose the only pointer to the old list.
- It works for an **empty list** too: `*headPtrPtr` is `NULL`, so the new node becomes the only node.
- It takes the same time for 3 or 3 million elements – **O(1)**.

What Happens in the Loop

- Let's explain what happens here:
- We start by assigning the first element to `currentPtr`.
- The loop continues **as long as** `currentPtr` is not `NULL`.
- Each iteration assigns `currentPtr` to `currentPtr->nextPtr`, i.e., moves to the next element.
- The last element of the list has its `nextPtr` set to `NULL`, so when we reach it, the condition fails and the loop stops.
- We must use the **`arrow operator ->`** instead of the dot (`.`) because we are accessing fields of a structure **through a pointer**. For example, these are equivalent:

```
currentPtr->value /* <===> */ (*currentPtr).value
```



WHY WE USE `currentPtr` INSTEAD OF `headPtr`

- *Why didn't we use `headPtr` directly in the loop?*
- Because if we overwrite `headPtr`, we lose access to the previous elements. That's why `headPtr` is usually reserved as a reference to the head of the list, used primarily for reading or accessing the start of the list.
- We use `currentPtr` as a temporary pointer to safely traverse the list.

THANK

You!